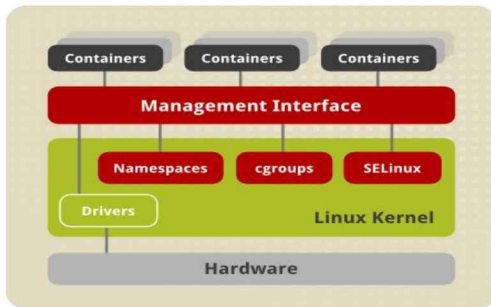
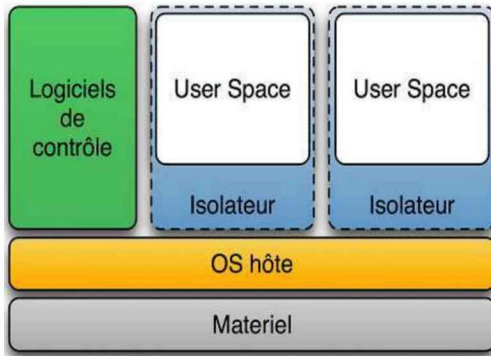


Les techniques de Virtualisation

Virtualisation au niveau du SE ou **Isolateur / cloisonnement**



- Isole l'exécution des applications dans des contextes d'exécution.
- Chaque instance d'espace utilisateur représente un environnement d'exécution virtuel autonome appelé **container**, **partition**, ou **jail**, selon la technologie utilisée (Docker, openVZ, LXC ...).
- Les environnements utilisateurs sont entièrement **cloisonnés**
- Exploite les **namespaces** et les **cgroups** du système linux
- séparer le contexte d'exécution de chaque application.
- Solution très performante **mais** isolation partielle
- Exemple : chroot, OpenVZ(Virtuozzo), Docker, LXC (Cgroups)

Les techniques de Virtualisation

Virtualisation au niveau du SE ou **Isolateur / cloisonnement**

namespaces:

Les espaces de noms (**namespaces**) sont une fonctionnalité Linux qui partitionne les ressources du noyau de telle sorte

- qu'un ensemble de processus voit un ensemble de ressources tandis qu'un autre ensemble de processus voit un ensemble différent de ressources.
- qu'un ensemble de processus sont séparés de telle façon qu'ils ne puissent pas « voir » les ressources des autres groupes. (on parle aussi de contextes)
- Par défaut, chaque système Linux a initialement un seul espace de noms. Toutes les ressources système, telles que les systèmes de fichiers, les ID de processus, les ID d'utilisateur, les interfaces réseau et autres, appartiennent à l'espace de noms unique.
- Il existe plusieurs types d'espaces de noms, de sorte qu'un processus n'appartient pas à un seul espace de noms, mais à un espace de nom de chaque type.
 - **Mount (mnt)** **Process ID (pid)**
 - **Network (net)**
 - **Inter-process communication (ipc)**
 - **User ID (user)**
 - **Control group (cgroup)**
[Mars 2016]

namespaces: Exemple de ressources

les **PID**, les **noms d'hôte**, les **UID**, les **noms de fichiers** et certains noms associés à l'accès au réseau et la communication interprocessus (**IPC**).

- L'espace de nommage par identifiant de processus (**PID namespace**) fournit l'isolation pour l'allocation des PIDs, la liste des processus et de leurs détails.
- L'espace de nommage réseau (**Network namespace**) isole le contrôleur de l'interface réseau (physique ou virtuel), les règles de pare-feu **iptables**, les tables de routage, etc. Les espaces de nommage réseau peuvent être connectés les uns avec chacun des autres en utilisant le périphérique virtuel Ethernet « **veth** ».
- L'espace de nommage « UTS » (**UTS namespace**) permet le changement de nom d'hôte.
- L'espace de nommage de montage (**Mount namespace**) permet de créer différents modèles de systèmes de fichiers, ou de créer certains points de montage en lecture-seule.
- L'espace de nommage IPC (**IPC namespace**) isole le système de communication inter-processus entre les espaces de nommage.

namespaces: Obectif

⇒ **Isolation par espace de nommage**

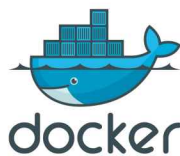
- Le rôle des namespaces est de créer un nouveau contexte système isolé pour le processus ciblé. On obtient une isolation pour :
 1. un nouvel espace de PID est initialisé et le processus prend le numéro de PID 1
 2. une nouvelle pile réseau est allouée au processus, donc aucun conflit possible avec les services réseaux de l'hôte
 3. un système de fichier indépendant permettant de monter/démonter les volumes sans incidence pour l'hôte

namespaces: Utilisation

Les espaces de noms sont un aspect fondamental des conteneurs sous Linux.

Divers logiciels de conteneur utilisent des espaces de noms Linux (namespaces) en combinaison avec des groupes de contrôle (**cgroups**) pour isoler leurs processus, notamment **Docker** et **LXC**.

Les espaces de nom sont créés notamment avec la commande « **unshare** ».



namespaces: Test

```
sudo unshare --fork --pid --mount-proc bash
```

```
ps x
```

PID	TTY	STAT	TIME	COMMAND
-----	-----	------	------	---------

1	pts/0	S	0:00	bash
9	pts/0	R+	0:00	ps x

```
# Côté hôte
```

```
ps faux
```

USER	PID	TTY	STAT	START	TIME	COMMAND
ahmed	27251	tty1	S	12:25	0:00	_ -bash
root	27281	tty1	S	12:26	0:00	_ sudo unshare ...
root	27282	tty1	S	12:26	0:00	_ unshare ...
root	27283	tty1	S+	12:26	0:00	_ bash

cgroups(control groups):

- **cgroups (control groups)** est une fonctionnalité du noyau Linux pour limiter, compter et isoler l'utilisation des ressources (processeur, mémoire, utilisation disque, etc.).
- Un cgroup est un groupe de contrôle de processus.
- Un groupe de contrôle est une suite de processus qui sont liés par le(s) même(s) critère(s).
- Ces groupes peuvent être organisés hiérarchiquement, de façon que chaque groupe hérite des limites de son groupe parent.

cgroups(control groups) v1: Contrôleurs

- Le noyau Linux fournit l'accès à plusieurs contrôleurs (sous-systèmes) à travers l'interface cgroup :
- **cpuset** : allocation de ressources CPU et mémoire vive
- **cpuacct** : permet de comptabiliser la consommation de cycle CPU
- **memory** : contrôle de la mémoire vive et du cache d'un groupe
- **devices** : autorise ou refuse l'accès à un périphérique
- **net_cls** : gère l'accès au réseau
- **blkio** : gère l'accès aux périphériques de type block (disque durs...)

cgroups(control groups): Objectif

- **Limitation des ressources** : des groupes peuvent être mis en place afin de ne pas dépasser une limite de mémoire
- **Priorisation** : certains groupes peuvent obtenir une plus grande part de ressources processeur ou de bande passante d'entrée-sortie
- **Comptabilité** : permet de mesurer la quantité de ressources consommées par certains systèmes en vue de leur facturation par exemple
- **Isolation** : séparation par espace de nommage pour les groupes, afin qu'ils ne puissent pas voir les processus des autres, leurs connexions réseaux ou leurs fichiers.
- **Contrôle** : figer les groupes ou créer un point de sauvegarde et redémarrer

cgroups(control groups): Test / installation

Il faut installer les paquets suivants :

- **cgroup-tools** - contrôle et surveillance des groupes de contrôle (outils)
- **libcgroup1** - control and monitor control groups (library)

\$ sudo apt install cgroup-tools

cgroups(control groups): Liste des contrôleurs

```
$ cat /proc/cgroups
```

#subsys_name	hierarchy	num_cgroups	enabled
cpuset	8	1	1
cpu	5	74	1
cpuacct	5	74	1
blkio	4	74	1
memory	6	112	1
devices	2	74	1
freezer	9	1	1
net_cls	7	1	1
perf_event	10	1	1
net_prio	7	1	1
...			

cgroups(control groups): Création des politiques (à la volée)

```
# Création d'une politique par défaut sans limite
```

```
$ sudo cgcreate -g cpu:/cpudefault
```

```
# Création d'une politique avec limite
```

```
$ sudo cgcreate -g cpu:/cpulimited
```

```
# Limite avec un ratio de consommation CPU de 2:1
```

```
# (valeur totale CPU = 1024 donc  $1024/2 = 512$ )
```

```
$ sudo cgset -r cpu.shares=512 cpulimited
```

```
# (valeur totale CPU = 0-10000 donc  $10000/2 = 5000$ )
```

```
$ sudo cgset -r cpu.weight=5000 cpulimited #cgroups v2
```

cgroups(control groups): Outil de test

L'outil **stress** permet de créer une charge de calcul sur un système.
L'option **-c (--cpu)** lance N travail(s) basé(s) sur **sqrt()**.

\$ sudo apt install stress

La commande **cgexec** permet de lancer un processus en le plaçant dans un groupe(-g). Le processus va hériter des limitations de ce groupe.

Exemple :

\$ sudo cgexec -g cpu:cpudefault stress -c 1 --timeout 200s &

cgroups(control groups): Test n° 1

On lance 4 fois : (avec la politique par défaut sans limite)

\$ sudo cgexec -g cpu:cpudefault stress -c 1 --timeout 200s &

Ou :

\$ for i in \$(seq 4); do (sudo cgexec -g cpu:cpudefault stress -c 1 --timeout 200s &) ; done

\$ top

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
26164	root	20	0	8240	92	0	R	25,2	0,0	0:04.21	stress
26167	root	20	0	8240	96	0	R	25,2	0,0	0:02.68	stress
26173	root	20	0	8240	92	0	R	25,2	0,0	0:01.95	stress
26170	root	20	0	8240	96	0	R	24,8	0,0	0:02.21	stress

On obtient une répartition équitable du CPU (autour de 25%).

\$ sudo killall stress

cgroups(control groups): Test n°2

On lance 1 fois : (avec la politique par défaut sans limite)

```
$ sudo cgexec -g cpu:cpudefault stress -c 1 --timeout 200s &
```

On lance 3 fois : (politique avec limite ratio 2:1)

```
$ for i in $(seq 3); do (sudo cgexec -g cpu:cpulimited stress -c 1 --timeout 200s &) ; done
```

```
$ top
```

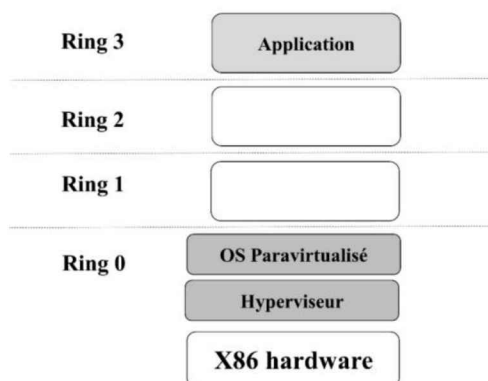
PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
26222	root	20	0	8240	96	0	R	66,3	0,0	0:13.17	stress
26231	root	20	0	8240	96	0	R	11,2	0,0	0:01.44	stress
26234	root	20	0	8240	96	0	R	11,2	0,0	0:01.42	stress
26235	root	20	0	8240	96	0	R	10,9	0,0	0:01.42	stress

Le ratio 2:1 sur 66% donne 33% réparti sur 3 processus soit 11% chacun.

```
$ sudo killall stress
```

Les techniques de Virtualisation

Para-virtualisation



- Dans cette technique, les appels système sont effectués par le noyau du système d'exploitation invité via des appels d'API, et ces appels sont directement traités par l'hyperviseur qui, à son tour, accomplit toutes les tâches.
- Les appels entre l'hyperviseur et le noyau du système d'exploitation invité sont appelés **hypercalls**.

- Red Hat VirtIO paravirtualisation pour KVM et Linux
- Xen
- Oracle VM Server pour SPARC

Nécessitent la modification des noyaux des SEs

Original (Non-Paravirtualized) Code:

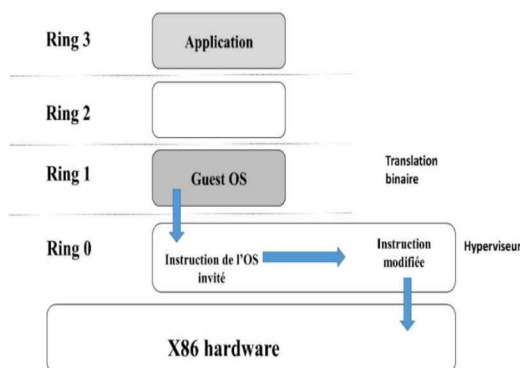
```
c
// Original code in the guest OS interacting directly with hardware
void read_data_from_disk(char* buffer, int sector_number) {
    // Assume direct hardware access
    // Read data from the specified sector on the disk
    // ...
}
```

Paravirtualized Code:

```
c
// Paravirtualized code in the guest OS using hypercalls
void read_data_from_disk_paravirtualized(char* buffer, int sector_number) {
    // Use hypercall to communicate with the hypervisor
    // The hypervisor handles the actual I/O operation
    hypervisor_hypercall(HYPERCALL_READ_DISK, buffer, sector_number);
}
```

Les techniques de Virtualisation

Virtualisation complète (Full Virtualization using **binary translation**)



- Parallels Desktop for Mac
- Parallels Workstation
- VMware ESXi

- Développée par **VMware en 1988**, la technique de **translation binaire** permet de placer les VMM dans le RING 0 et de déplacer les SE hébergés dans le RING 1 ce qui garantit un niveau de privilège supérieur à celui des processus s'exécutant dans le RING 3
- Le SE invité n'ont pas conscience qu'il s'exécute sur une VM.
- Les appels sont généralement contrôlés et surveillés par l'hyperviseur qui fournit une émulation de CPU pour gérer et modifier les privilèges.
- Les **instructions critiques** sont capturées et traduites à l'aide de la « **binary translation** ».
- Mais ce scénario n'est pas très efficace et entraîne une dégradation des performances du système par rapport à la **paravirtualisation**.

Original x86 Assembly Code:

assembly

 Copy code

```
; Original x86 assembly code in the guest OS
mov eax, 10    ; Move the value 10 into the EAX register
add eax, ebx   ; Add the value in EBX to EAX
```

Binary Translation to RISC-V:

assembly

 Copy code

```
; Translated RISC-V assembly code
li t0, 10      ; Load the immediate value 10 into register t0
add t0, t0, t1 ; Add the value in t1 to t0
```

Original x86 Assembly Code:

assembly

 Copy code

```
; Original x86 assembly code in the guest OS
pop eax          ; Pop a value from the stack into EAX
mov ebx, eax     ; Move the value in EAX to EBX
```

Translated x86 Assembly Code using Binary Translation:

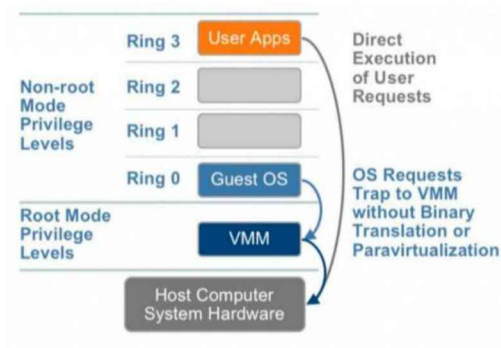
assembly

 Copy code

```
; Translated x86 assembly code for a virtualized environment
sub esp, 4      ; Adjust the stack pointer to simulate the pop operation
mov eax, [esp]  ; Move the value from the emulated stack top into EAX
mov ebx, eax    ; Move the value in EAX to EBX
```

Les techniques de Virtualisation

Virtualisation assistée par le matériel (**Hardware assisted virtualization**)



- Avec l'arrivée des derniers processeurs sur le marché, **INTEL (VT-x)** et **AMD (AMD-V)** ont conçu de nouveaux processeurs qui permettent de fournir une couche supplémentaire au-dessus de **RING 0** qui aide l'hyperviseur à fonctionner et à prendre le contrôle du système d'exploitation invité.
- le système d'exploitation invité a un accès direct aux ressources
- Cela élimine la surcharge de l'émulation des instructions de CPU.

Les domaines de la virtualisation

La virtualisation matérielle

- Machine virtuelle
- Virtualisation des serveurs
- Hyperviseur

Virtualisation du réseau

- Le réseau local virtuel (VLAN)
- Software Defined Networking (SDN)

Virtualisation logicielle

- Virtualisation des applications
- Virtualisation de postes de travail

Virtualisation du stockage

- Direct Attached Storage(DAS)
- Storage Area Network(SAN)
- Network Attached Storage(NAS)